Identificación y análisis de estructuras de decisión

Revelado: organizador de fotos, videos y audios por fecha en Python

Estudiante	Andrés Felipe Martínez Chamucero
Programa	Especialización en Diseño y Creación con IA
Asignatura	Lenguajes de Programación Multiplataforma
Docente	_____
Fecha	Noviembre de 2026

1. Situación problema

En el cubrimiento de eventos y la producción audiovisual se acumulan cientos de archivos en poco tiempo: fotografías de cámara en **.jpg** y en formato **RAW**, fotos de iPhone en **.heic**, capturas y diseños en **.png**, videos en **.mov** y **.mp4**, y entrevistas o sonido ambiente en **.wav** y **.mp3**. Los dispositivos los nombran con códigos como *IMG_4521.JPG* o *VID_20261101.mp4*, que no indican a qué día pertenecen, y encontrar después el material de una fecha concreta toma mucho tiempo.

Enunciado

Diseñe un algoritmo que, a partir de una carpeta elegida por el usuario, organice automáticamente sus fotos y videos por fecha. Debe reconocer fotos (.jpg, .jpeg, .png, .heic, .heif), fotos RAW de las principales marcas de cámaras (.cr2, .cr3, .nef, .arw, .dng, .raf, .orf, .rw2), videos (.mov, .mp4) y audios (.mp3, .wav). Para cada uno debe averiguar la fecha en que se tomó o grabó, crear una carpeta por día con el nombre **AAAAMMDD** (año, mes y día, sin guiones) y renombrar el archivo como **AAAAMMDD (1).ext**. Dentro de cada día se ordenan primero las fotos JPG, luego HEIC, PNG, RAW, videos y audios, y la numeración es una sola secuencia continua: si un día tiene tres fotos y dos videos, las fotos quedan como (1), (2) y (3) y los videos como (4) y (5). Los demás archivos no se modifican. El programa debe trabajar sobre los archivos del computador sin que haya que subirlos a ningún servicio, mostrar una vista previa y pedir confirmación antes de cambiar cualquier nombre, no sobrescribir nunca un archivo existente y dejar un registro de los nombres originales.

2. Planteamiento de la solución

Entradas	La carpeta elegida por el usuario y su respuesta de confirmación (s/n).
Proceso	Validar la carpeta (while); recorrer los archivos (for); clasificar por extensión en seis tipos (if/elif/else); obtener la fecha según el tipo (if/elif + while para leer videos y WAV); ordenar por día y prioridad del tipo (JPG, HEIC, PNG, RAW, video, audio); calcular el nombre nuevo con un contador por día y evitar duplicados (while); confirmar (while + if/else); renombrar, mover y registrar (for).
Salidas	Carpetas por día con los archivos renombrados, un resumen en pantalla y el archivo *registro_organizador.csv* con los nombres originales.

¿De dónde sale la fecha de cada archivo?

Tipo	Extensiones	Fuente de la fecha
Foto	.jpg, .jpeg, .png, .heic, .heif	Fecha de toma que la cámara o el celular guardan dentro de la imagen (datos EXIF).

RAW	.cr2, .cr3, .nef, .arw, .dng, .raf, .orf, .rw2	Fecha de toma (EXIF). El CR3 de Canon tiene la misma estructura de un video, así que si no hay EXIF se lee como video.
Video	.mov, .mp4	Fecha de creación guardada en el bloque *mvhd* del video, en hora UTC, convertida a la hora de Colombia.
Audio	.wav, .mp3	WAV: bloque *bext* que escriben las grabadoras de campo. MP3: etiqueta de fecha ID3 (TDRC).
Respaldo	Cualquiera	Si el archivo no trae la fecha por dentro (capturas de pantalla, fotos de WhatsApp), se usa la fecha de modificación del archivo.

¿Cómo se da el permiso sin subir los archivos?

La aplicación no sube nada a internet: se ejecuta en el propio computador y solo puede tocar la carpeta que el usuario le entregue. El permiso funciona en tres niveles:

- **Elección de la carpeta.** Al iniciar, se abre la ventana del sistema para seleccionar una carpeta (o se escribe la ruta al ejecutar el programa). Esa elección es el permiso: el programa trabaja únicamente ahí y no entra en las subcarpetas.
- **Vista previa y confirmación.** Antes de cambiar algo, el programa muestra cómo quedará cada archivo y pregunta «¿Organizar...? (s/n)». Si la respuesta es «n», no se modifica nada.
- **Reversibilidad.** Los archivos solo se renombran y se mueven, nunca se borran ni se sobrescriben, y el registro CSV permite saber cuál era el nombre original de cada uno.

3. Pseudocódigo

```
Algoritmo Revelado_OrganizadorPorFecha
    Definir carpeta, ext, tipo, dia, nuevo, respuesta Como Cadena
    Definir omitidos, numero Como Entero
    Definir elementos Como Lista
    Definir conteo, contador_dia Como Diccionario

    // ---------- Paso 1: el usuario da el permiso eligiendo la carpeta ----------
    carpeta <- carpeta elegida en la ventana del sistema
    Mientras carpeta no es una carpeta válida Hacer
        Escribir "No es una carpeta válida"
        Leer carpeta
    FinMientras

    // ---------- Paso 2: recorrer, clasificar y leer la fecha ----------
    elementos <- []; omitidos <- 0
    conteo <- {jpg:0, heic:0, png:0, raw:0, video:0, audio:0}
    Para cada archivo en carpeta Hacer
        Si archivo es una subcarpeta Entonces
            Continuar                              // no se toca
        FinSi
        tipo <- TipoArchivo(Minusculas(extensión de archivo))
        Si tipo = Nulo Entonces
            omitidos <- omitidos + 1               // .pdf, .txt, etc.
            Continuar
        FinSi
        conteo[tipo] <- conteo[tipo] + 1
        fecha, origen <- FechaInterna(archivo)
        Si fecha = Nulo Entonces                   // respaldo
            fecha <- fecha de modificación del archivo
            origen <- "modificación"
        FinSi
        Agregar (archivo, fecha, tipo, origen) a elementos
    FinPara
    Escribir conteo, omitidos
    Si elementos está vacía Entonces
        Escribir "No hay archivos para organizar"; Terminar
    FinSi

    // ---------- Paso 3: ordenar y calcular el nombre nuevo ----------
    Ordenar elementos por (día, Prioridad(tipo), fecha)
    contador_dia <- {}
    Para cada e en elementos Hacer
        dia <- Formato(e.fecha, "AAAAMMDD")
        numero <- contador_dia[dia] + 1            // la secuencia continúa
        nuevo <- dia + " (" + numero + ")" + ext
        Mientras carpeta/dia/nuevo ya existe Hacer  // nunca sobrescribir
            numero <- numero + 1
            nuevo <- dia + " (" + numero + ")" + ext
        FinMientras
        contador_dia[dia] <- numero
        e.dia <- dia; e.nuevo <- nuevo
    FinPara

    // ---------- Paso 4: vista previa y confirmación ----------
    Para cada e en elementos Hacer
        Escribir e.nombre, " -> ", e.dia, "/", e.nuevo, " (", e.origen, ")"
    FinPara
    respuesta <- ""
```

```
    Mientras respuesta <> "s" Y respuesta <> "n" Hacer
        Escribir "¿Organizar los archivos? (s/n)"; Leer respuesta
    FinMientras

    // ---------- Paso 5: renombrar, mover y registrar ----------
    Si respuesta = "n" Entonces
        Escribir "Operación cancelada. No se modificó ningún archivo"
    SiNo
        Para cada e en elementos Hacer
            Crear la carpeta e.dia si no existe
            Renombrar e.archivo como carpeta/e.dia/e.nuevo
            Escribir en registro_organizador.csv: nombre original, nombre nuevo
        FinPara
        Escribir resumen
    FinSi
FinAlgoritmo

Funcion tipo <- TipoArchivo(ext)
    Si ext en [".jpg", ".jpeg"] Entonces          tipo <- "jpg"
    SiNo Si ext en [".heic", ".heif"] Entonces    tipo <- "heic"
    SiNo Si ext = ".png" Entonces                 tipo <- "png"
    SiNo Si ext en [".cr2", ".cr3", ".nef", ".arw", ".dng",
                    ".raf", ".orf", ".rw2"] Entonces  tipo <- "raw"
    SiNo Si ext en [".mov", ".mp4"] Entonces      tipo <- "video"
    SiNo Si ext en [".mp3", ".wav"] Entonces      tipo <- "audio"
    SiNo                                          tipo <- Nulo
    FinSi
FinFuncion

Funcion p <- Prioridad(tipo)       // jpg 0, heic 1, png 2, raw 3, video 4, audio 5
    Si tipo = "jpg" Entonces p <- 0
    SiNo Si tipo = "heic" Entonces p <- 1
    SiNo Si tipo = "png" Entonces p <- 2
    SiNo Si tipo = "raw" Entonces p <- 3
    SiNo Si tipo = "video" Entonces p <- 4
    SiNo p <- 5
    FinSi
FinFuncion

Funcion fecha, origen <- FechaInterna(archivo)
    tipo <- TipoArchivo(ext); fecha <- Nulo
    Si tipo en ["jpg", "heic", "png"] Entonces
        fecha <- fecha de toma (EXIF); origen <- "EXIF"
    SiNo Si tipo = "raw" Entonces
        fecha <- fecha de toma (EXIF); origen <- "EXIF"
        Si fecha = Nulo Y ext = ".cr3" Entonces
            fecha <- LeerAtomoMvhd(archivo); origen <- "metadatos"
        FinSi
    SiNo Si tipo = "video" Entonces
        fecha <- LeerAtomoMvhd(archivo); origen <- "metadatos"
    SiNo Si tipo = "audio" Entonces
        Si ext = ".wav" Entonces
            Mientras queden bloques RIFF Hacer          // buscar 'bext'
                Si bloque = "bext" Entonces fecha <- fecha de grabación FinSi
            FinMientras
        SiNo
            fecha <- etiqueta ID3 de fecha (TDRC)
        FinSi
```

```
        origen <- "metadatos"
    FinSi
FinFuncion

Funcion fecha <- LeerAtomoMvhd(archivo)
    pos <- 0
    Mientras pos < tamaño del archivo Hacer             // recorrer átomos
        Leer tamaño y tipo del átomo en pos
        Si tipo = "moov" Entonces       pos <- pos + cabecera   // entrar
        SiNo Si tipo = "mvhd" Entonces  fecha <- segundos desde 1904 (UTC) a hora de Colombia
                                        Salir del ciclo
        SiNo                            pos <- pos + tamaño     // saltar
        FinSi
    FinMientras
FinFuncion
```

4. Diagrama de flujo

Convenciones: óvalo = inicio/fin; rectángulo = proceso; rectángulo doble = subproceso; rombo = decisión; paralelogramo = entrada o salida; círculo con letra = conector (el flujo continúa en el conector de la misma letra).

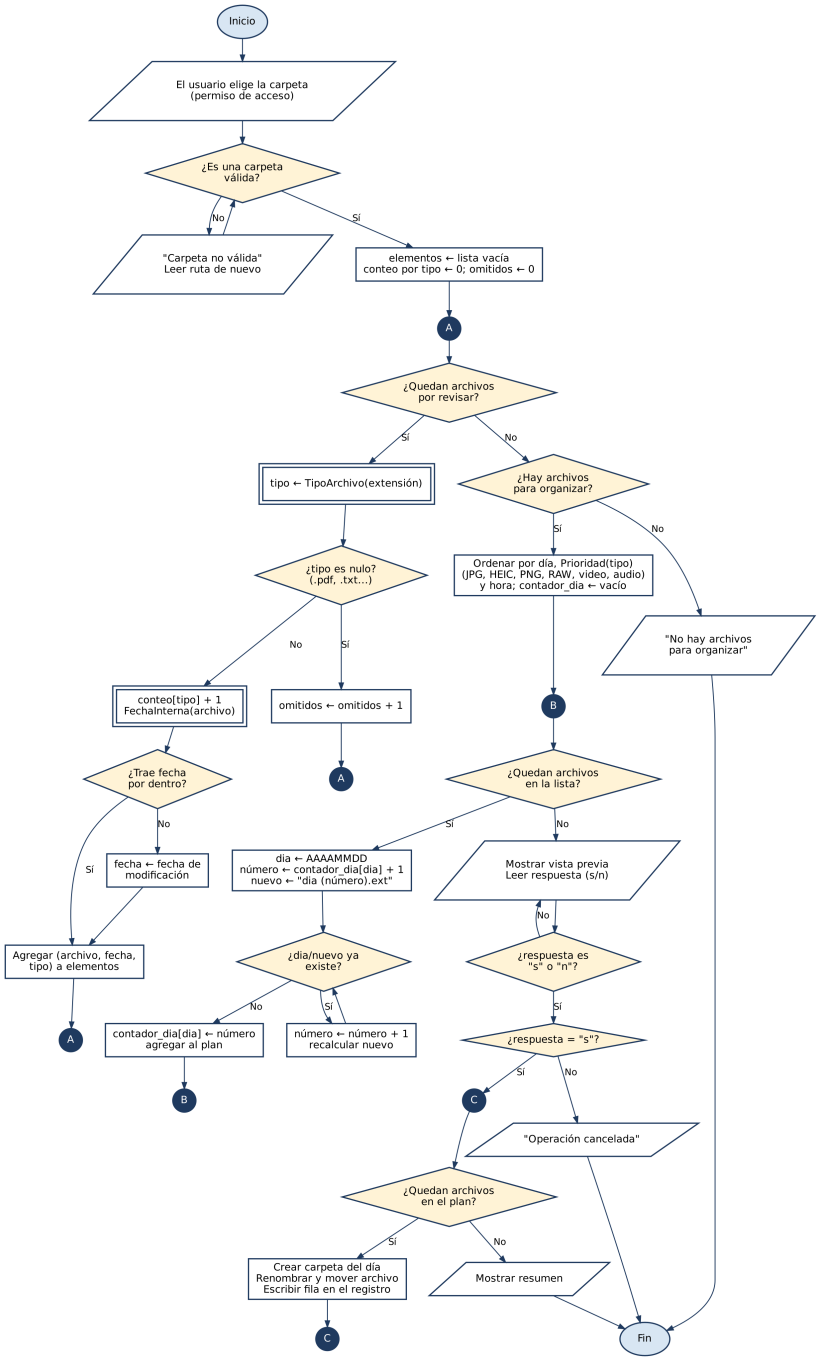


Figura 1. Diagrama de flujo del algoritmo principal.

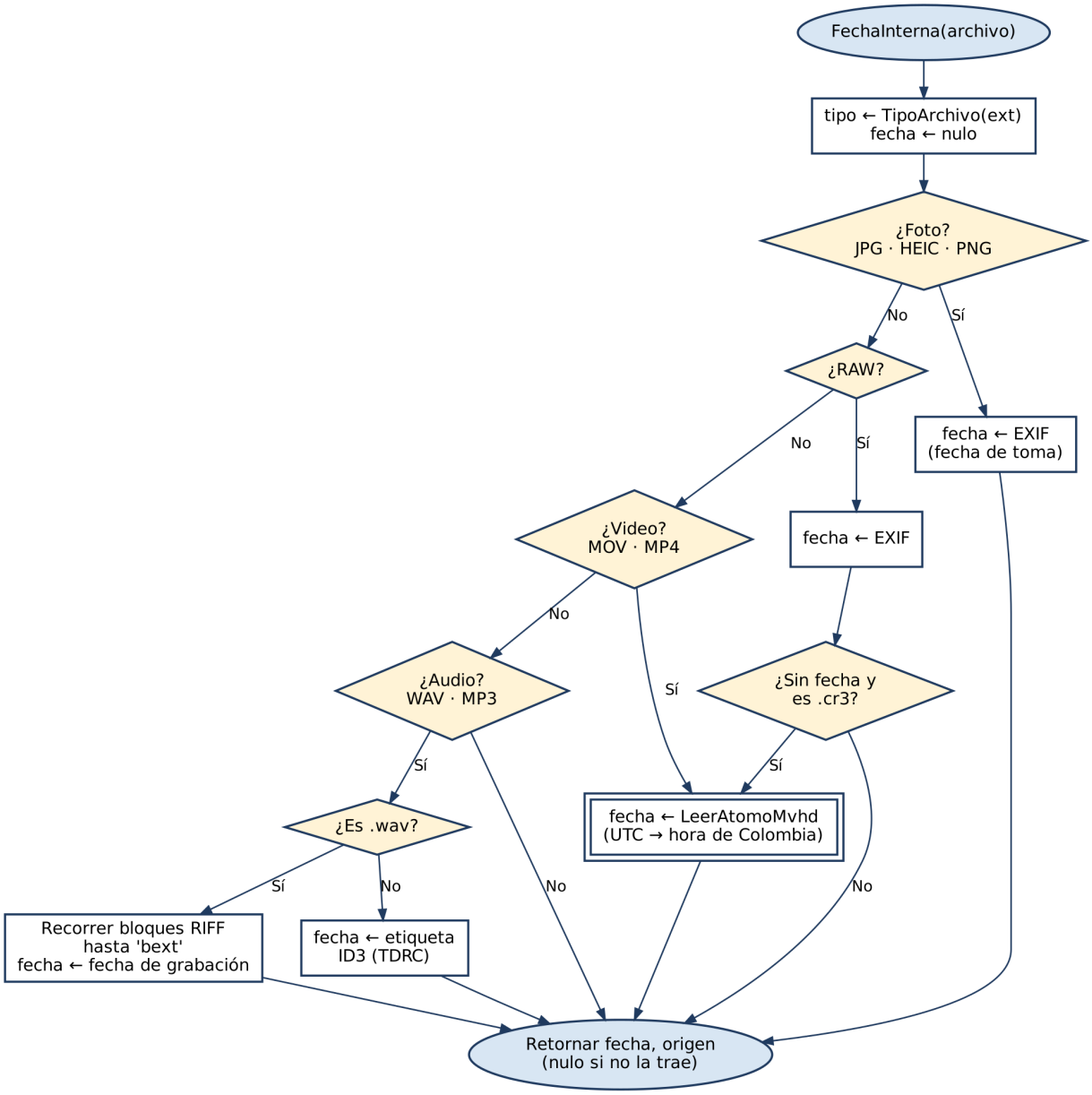


Figura 2. Diagrama de flujo del subproceso FechaInterna.

5. Código fuente en Python (comentado)

Archivo *organizador_fotos.py*. La lógica funciona con la librería estándar de Python; las librerías opcionales *exifread*, *Pillow*, *pillow-heif* y *mutagen* permiten leer la fecha de fotos, HEIC, RAW y MP3 (*pip install -r requirements.txt*). La app web (*app.py*) importa estas mismas funciones.

```
# ============================================================
# Revelado · Organizador de fotos, videos y audios por fecha
# Taller: Identificación y análisis de estructuras de decisión
# Autor: Andrés Felipe Martínez Chamucero
# ------------------------------------------------------------
# Versión para ejecutar en el computador. El usuario elige una
# carpeta (ese es el permiso que le da a la app) y el algoritmo:
#   1. Recorre los archivos y reconoce fotos (JPG, PNG, HEIC),
#      fotos RAW, videos (MOV, MP4) y audios (MP3, WAV).
#   2. Averigua la fecha en que se tomó o grabó cada uno.
#   3. Muestra una vista previa y pide confirmación.
#   4. Crea una carpeta por día (AAAAMMDD) y renombra cada
#      archivo como "AAAAMMDD (1).ext". Dentro de cada día van
#      primero las JPG, luego HEIC, PNG, RAW, videos y audios,
#      con una sola numeración continua.
#   5. Guarda un registro CSV con los nombres originales.
# Este mismo archivo lo usa la app web (app.py) para leer las
# fechas y calcular los nombres nuevos.
# Estructuras usadas: if / elif / else, for, while
# Librerías opcionales: exifread, Pillow, pillow-heif, mutagen
# ============================================================

import csv
import datetime
import logging
import os
import struct
import sys
from pathlib import Path

# ---------- Formatos reconocidos ----------

EXT_JPG = {".jpg", ".jpeg"}
EXT_HEIC = {".heic", ".heif"}                       # fotos de iPhone
EXT_PNG = {".png"}
EXT_RAW = {".cr2", ".cr3", ".nef", ".arw", ".dng",  # Canon, Nikon, Sony,
           ".raf", ".orf", ".rw2"}                  # Fuji, Olympus, Panasonic
EXT_VIDEO = {".mov", ".mp4"}
EXT_AUDIO = {".mp3", ".wav"}
EXT_TODAS = EXT_JPG | EXT_HEIC | EXT_PNG | EXT_RAW | EXT_VIDEO | EXT_AUDIO

ZONA_LOCAL = datetime.timezone(datetime.timedelta(hours=-5))   # hora de Colombia
logging.getLogger("exifread").setLevel(logging.CRITICAL)        # sin avisos técnicos


def tipo_archivo(extension):
    """Clasifica un archivo según su extensión (en minúsculas)."""
    if extension in EXT_JPG:
        return "jpg"
    elif extension in EXT_HEIC:
```

```
        return "heic"
    elif extension in EXT_PNG:
        return "png"
    elif extension in EXT_RAW:
        return "raw"
    elif extension in EXT_VIDEO:
        return "video"
    elif extension in EXT_AUDIO:
        return "audio"
    else:
        return None                                  # no se organiza


def prioridad(tipo):
    """Orden dentro de cada día: JPG, HEIC, PNG, RAW, videos, audios."""
    if tipo == "jpg":
        return 0
    elif tipo == "heic":
        return 1
    elif tipo == "png":
        return 2
    elif tipo == "raw":
        return 3
    elif tipo == "video":
        return 4
    else:
        return 5                                     # audio


# ---------- Lectura de fechas (funcionan con cualquier archivo abierto) ----------

def convertir_texto_fecha(texto):
    """Convierte textos como '2026:11:02 19:05:30' o '2026-11-02T19:05' en fecha."""
    texto = str(texto).strip().replace("T", " ").split(".")[0].split("+")[0]
    formatos = ["%Y:%m:%d %H:%M:%S", "%Y-%m-%d %H:%M:%S", "%Y-%m-%d %H:%M",
                "%Y:%m:%d", "%Y-%m-%d"]
    for formato in formatos:
        try:
            fecha = datetime.datetime.strptime(texto[:19], formato)
            if fecha.year >= 1990:                   # descarta fechas vacías
                return fecha
        except ValueError:
            continue
    return None


def fecha_foto(archivo):
    """Fecha de toma (EXIF) de JPG, PNG, HEIC y RAW."""
    # Intento 1: exifread, que entiende JPG, HEIC y la mayoría de RAW
    try:
        import exifread
        archivo.seek(0)
        etiquetas = exifread.process_file(archivo, details=False,
                                          stop_tag="EXIF DateTimeOriginal")
        for clave in ("EXIF DateTimeOriginal", "Image DateTime"):
            if clave in etiquetas:
                fecha = convertir_texto_fecha(etiquetas[clave])
                if fecha:
                    return fecha
    except Exception:
```

```
        pass
    # Intento 2: Pillow (y pillow-heif para HEIC)
    try:
        from PIL import Image
        try:
            import pillow_heif
            pillow_heif.register_heif_opener()
        except ImportError:
            pass
        archivo.seek(0)
        with Image.open(archivo) as img:
            exif = img.getexif()
            valor = exif.get_ifd(0x8769).get(36867) or exif.get(306)
        if valor:
            return convertir_texto_fecha(valor)
    except Exception:
        pass
    return None


def fecha_video(archivo):
    """Fecha de creación de MP4, MOV (y CR3), guardada en el átomo 'mvhd'."""
    try:
        archivo.seek(0, os.SEEK_END)
        fin = archivo.tell()
        pos = 0
        while pos < fin:                            # recorre los átomos
            archivo.seek(pos)
            cabecera = archivo.read(8)
            if len(cabecera) < 8:
                break
            tamano, tipo = struct.unpack(">I4s", cabecera)
            largo_cab = 8
            if tamano == 1:
                tamano = struct.unpack(">Q", archivo.read(8))[0]
                largo_cab = 16
            elif tamano == 0:
                tamano = fin - pos

            if tipo == b"moov":                     # entrar a 'moov'
                pos += largo_cab
                continue
            elif tipo == b"mvhd":                   # aquí está la fecha
                version = archivo.read(4)[0]
                if version == 1:
                    segundos = struct.unpack(">Q", archivo.read(8))[0]
                else:
                    segundos = struct.unpack(">I", archivo.read(4))[0]
                if segundos == 0:
                    return None
                # Segundos desde 1904 en UTC -> hora de Colombia (UTC-5)
                utc = (datetime.datetime(1904, 1, 1, tzinfo=datetime.timezone.utc)
                       + datetime.timedelta(seconds=segundos))
                return utc.astimezone(ZONA_LOCAL).replace(tzinfo=None)

            if tamano < largo_cab:
                break
            pos += tamano
    except (OSError, struct.error, IndexError):
        pass
```

```
    return None


def fecha_wav(archivo):
    """Fecha de grabación de un WAV (bloque 'bext' de grabadoras como Zoom o Tascam)."""
    try:
        archivo.seek(0)
        if archivo.read(4) != b"RIFF":
            return None
        archivo.seek(12)
        while True:                                  # recorre los bloques
            cabecera = archivo.read(8)
            if len(cabecera) < 8:
                break
            nombre, tamano = struct.unpack("<4sI", cabecera)
            if nombre == b"bext":
                datos = archivo.read(tamano)
                fecha = datos[320:330].decode("ascii", "ignore")   # AAAA-MM-DD
                hora = datos[330:338].decode("ascii", "ignore")    # HH:MM:SS
                for separador in (":", "_", ".", " "):     # la norma admite
                    fecha = fecha.replace(separador, "-")  # varios separadores
                for separador in ("-", "_", ".", " "):
                    hora = hora.replace(separador, ":")
                return convertir_texto_fecha(f"{fecha} {hora}")
            archivo.seek(tamano + (tamano % 2), os.SEEK_CUR)
    except (OSError, struct.error):
        pass
    return None


def fecha_mp3(archivo):
    """Fecha de grabación guardada en las etiquetas ID3 de un MP3."""
    try:
        from mutagen.id3 import ID3
        archivo.seek(0)
        etiquetas = ID3(archivo)
        for clave in ("TDRC", "TDOR", "TDRL"):
            if clave in etiquetas:
                texto = str(etiquetas[clave].text[0])
                if len(texto) >= 10:                 # solo si trae día completo
                    return convertir_texto_fecha(texto)
    except Exception:
        pass
    return None


def fecha_interna(nombre, archivo):
    """Decide de dónde leer la fecha según el tipo de archivo.

    Devuelve (fecha, origen). Si el archivo no trae la fecha por
    dentro, devuelve (None, "") y quien llama decide el respaldo.
    """
    extension = Path(nombre).suffix.lower()
    tipo = tipo_archivo(extension)
    fecha = None
    origen = ""

    if tipo in ("jpg", "heic", "png"):
        fecha = fecha_foto(archivo)
        origen = "EXIF"
```

```
    elif tipo == "raw":
        fecha = fecha_foto(archivo)
        origen = "EXIF"
        if fecha is None and extension == ".cr3":    # el CR3 es como un video
            fecha = fecha_video(archivo)
            origen = "metadatos"
    elif tipo == "video":
        fecha = fecha_video(archivo)
        origen = "metadatos"
    elif tipo == "audio":
        if extension == ".wav":
            fecha = fecha_wav(archivo)
        else:
            fecha = fecha_mp3(archivo)
        origen = "metadatos"

    if fecha is None:
        origen = ""
    return fecha, origen


# ---------- Cálculo de los nombres nuevos ----------

def planificar(elementos, existe=None):
    """Ordena y asigna el nombre nuevo a cada elemento.

    elementos: lista de diccionarios con 'nombre', 'fecha' y 'tipo'.
    existe: función opcional que dice si una ruta ya está ocupada.
    Agrega a cada elemento 'dia' (AAAAMMDD) y 'nuevo' ("AAAAMMDD (n).ext").
    """
    elementos.sort(key=lambda e: (e["fecha"].date(), prioridad(e["tipo"]), e["fecha"]))
    contador_dia = {}
    for e in elementos:
        dia = e["fecha"].strftime("%Y%m%d")
        numero = contador_dia.get(dia, 0) + 1        # la secuencia continúa
        extension = Path(e["nombre"]).suffix.lower().replace(".jpeg", ".jpg")
        nuevo = f"{dia} ({numero}){extension}"
        while existe is not None and existe(dia, nuevo):   # nunca sobrescribir
            numero += 1
            nuevo = f"{dia} ({numero}){extension}"
        contador_dia[dia] = numero
        e["dia"] = dia
        e["nuevo"] = nuevo
    return elementos


# ---------- Programa para el computador ----------

def elegir_carpeta():
    if len(sys.argv) > 1:                            # ruta escrita al ejecutar
        return Path(sys.argv[1])
    try:                                             # ventana del sistema
        import tkinter
        from tkinter import filedialog
        ventana = tkinter.Tk()
        ventana.withdraw()
        ruta = filedialog.askdirectory(title="Elige la carpeta de fotos, videos y audios")
        ventana.destroy()
        if ruta:
            return Path(ruta)
```

```
    except Exception:
        pass
    return Path(input("Escribe la ruta de la carpeta: ").strip().strip('"'))


def main():
    print("=== REVELADO · ORGANIZADOR POR FECHA ===")
    carpeta = elegir_carpeta()
    while not carpeta.is_dir():                      # validar la carpeta
        print(f"  '{carpeta}' no es una carpeta válida.")
        carpeta = Path(input("Escribe la ruta de la carpeta: ").strip().strip('"'))
    print(f"Carpeta elegida: {carpeta}\n")

    # Paso 2: recorrer, clasificar y leer la fecha
    elementos = []
    conteo = {"jpg": 0, "heic": 0, "png": 0, "raw": 0, "video": 0, "audio": 0}
    omitidos = 0
    for ruta in sorted(carpeta.iterdir()):
        if not ruta.is_file():
            continue                                 # las subcarpetas no se tocan
        tipo = tipo_archivo(ruta.suffix.lower())
        if tipo is None:
            omitidos += 1                            # .pdf, .txt, etc.
            continue
        conteo[tipo] += 1
        with open(ruta, "rb") as archivo:
            fecha, origen = fecha_interna(ruta.name, archivo)
        if fecha is None:                            # respaldo: fecha del archivo
            fecha = datetime.datetime.fromtimestamp(ruta.stat().st_mtime)
            origen = "modificación"
        elementos.append({"nombre": ruta.name, "ruta": ruta, "fecha": fecha,
                          "tipo": tipo, "origen": origen})

    fotos = conteo["jpg"] + conteo["heic"] + conteo["png"]
    print(f"Encontrados: {fotos} fotos, {conteo['raw']} RAW, {conteo['video']} videos, "
          f"{conteo['audio']} audios, {omitidos} omitidos")
    if len(elementos) == 0:
        print("No hay archivos para organizar.")
        return

    # Paso 3: calcular nombres nuevos
    planificar(elementos, existe=lambda dia, nuevo: (carpeta / dia / nuevo).exists())
    dias = sorted({e["dia"] for e in elementos})

    print("\nVista previa:")
    for e in elementos:
        print(f"  {e['nombre']:<24} -> {e['dia']}/{e['nuevo']:<18} ({e['origen']})")

    # Paso 4: confirmación
    respuesta = ""
    while respuesta not in ("s", "n"):
        respuesta = input(f"\n¿Organizar {len(elementos)} archivos en {len(dias)} "
                          f"carpetas? (s/n): ").strip().lower()
    if respuesta == "n":
        print("Operación cancelada. No se modificó ningún archivo.")
        return

    # Paso 5: mover, renombrar y registrar
    registro = carpeta / "registro_organizador.csv"
    nuevo_registro = not registro.exists()
```

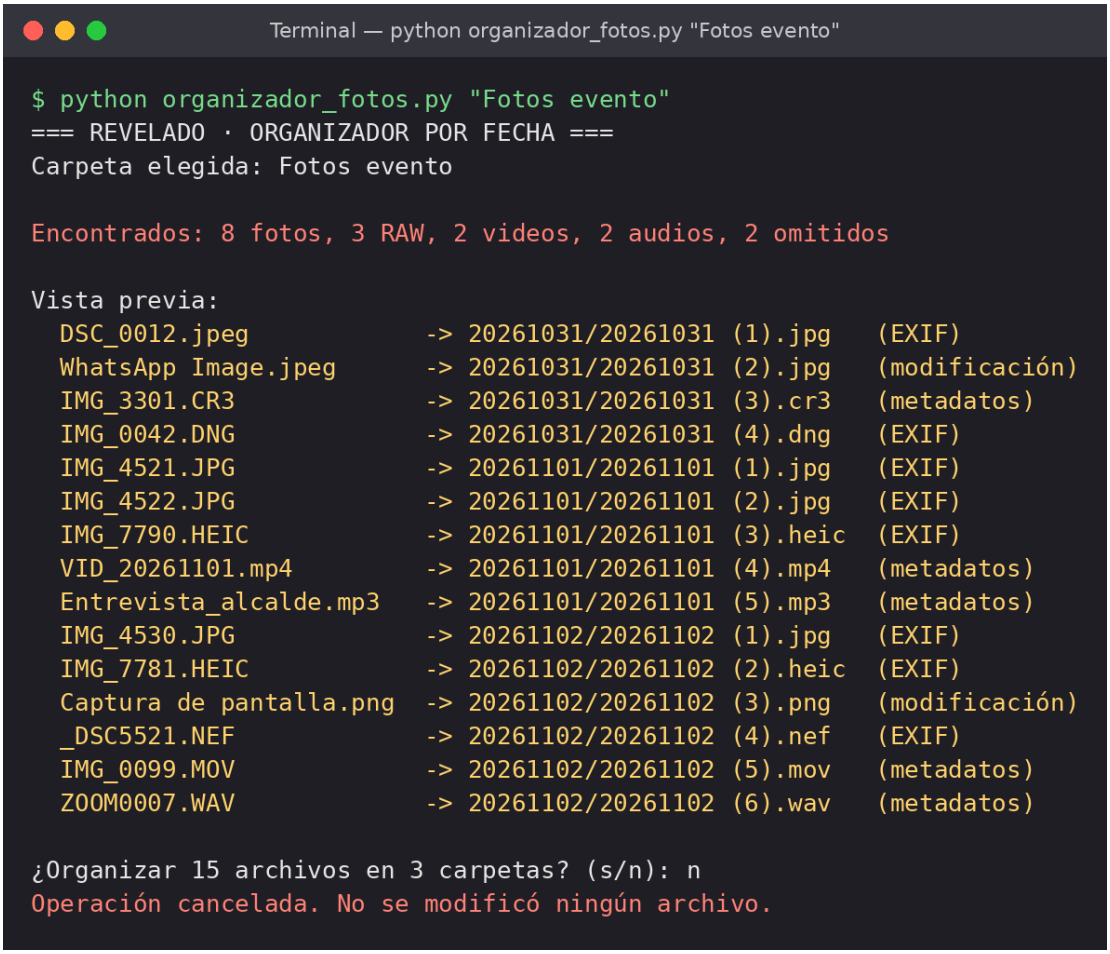
```
    movidos = errores = 0
    with open(registro, "a", newline="", encoding="utf-8") as archivo_csv:
        escritor = csv.writer(archivo_csv)
        if nuevo_registro:
            escritor.writerow(["nombre_original", "nombre_nuevo", "fuente_fecha"])
        for e in elementos:
            destino = carpeta / e["dia"] / e["nuevo"]
            try:
                destino.parent.mkdir(exist_ok=True)
                e["ruta"].rename(destino)
                escritor.writerow([e["nombre"], f"{e['dia']}/{e['nuevo']}", e["origen"]])
                movidos += 1
            except OSError as error:
                print(f"  No se pudo mover {e['nombre']}: {error}")
                errores += 1

    print("\n========== RESUMEN ==========")
    print(f"Archivos organizados: {movidos}")
    print(f"Carpetas por día: {', '.join(dias)}")
    if errores > 0:
        print(f"Archivos con error: {errores}")
    print(f"Registro guardado en: {registro.name}")

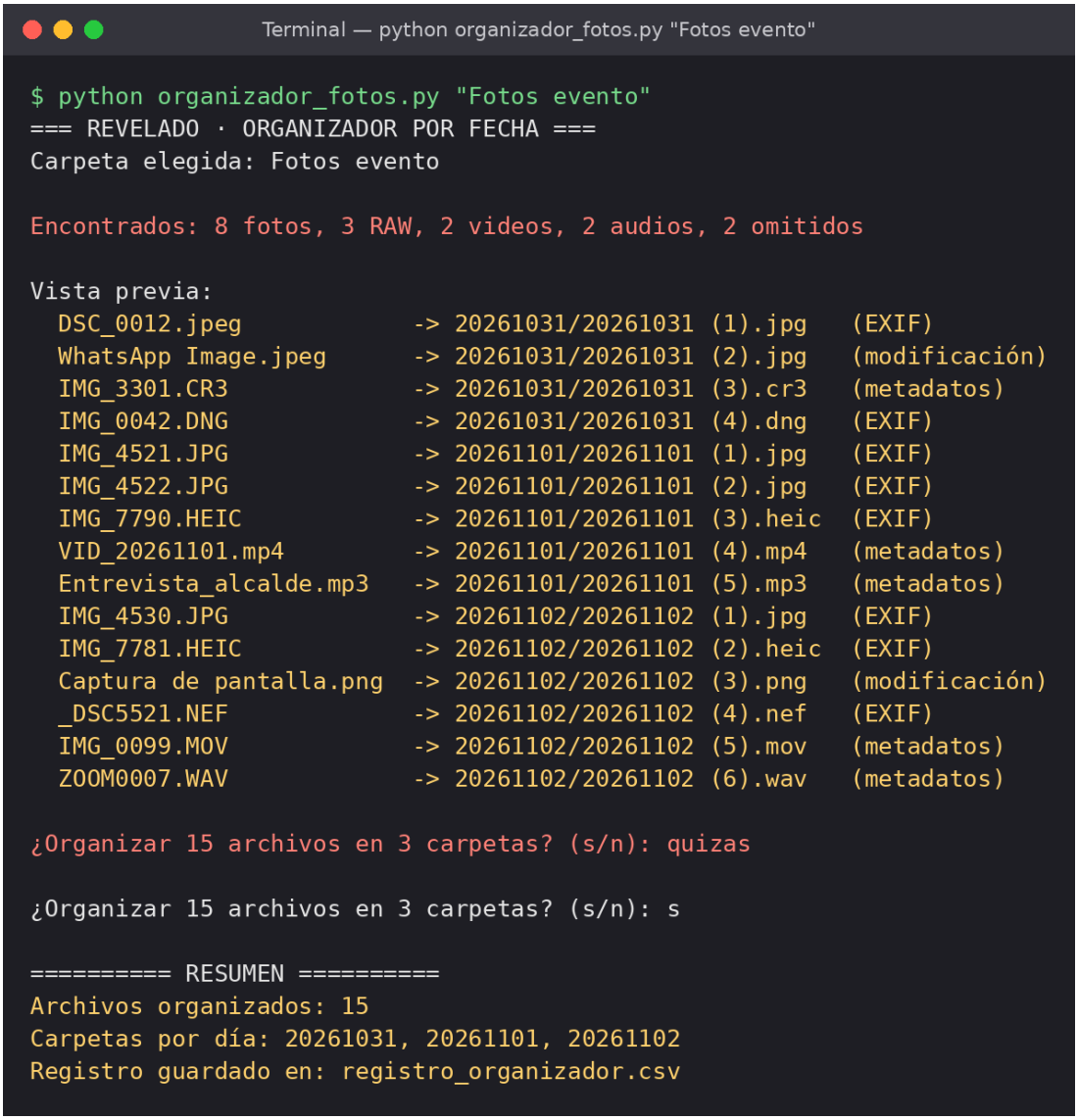

if __name__ == "__main__":
    main()
```

6. Capturas de pantalla de la ejecución

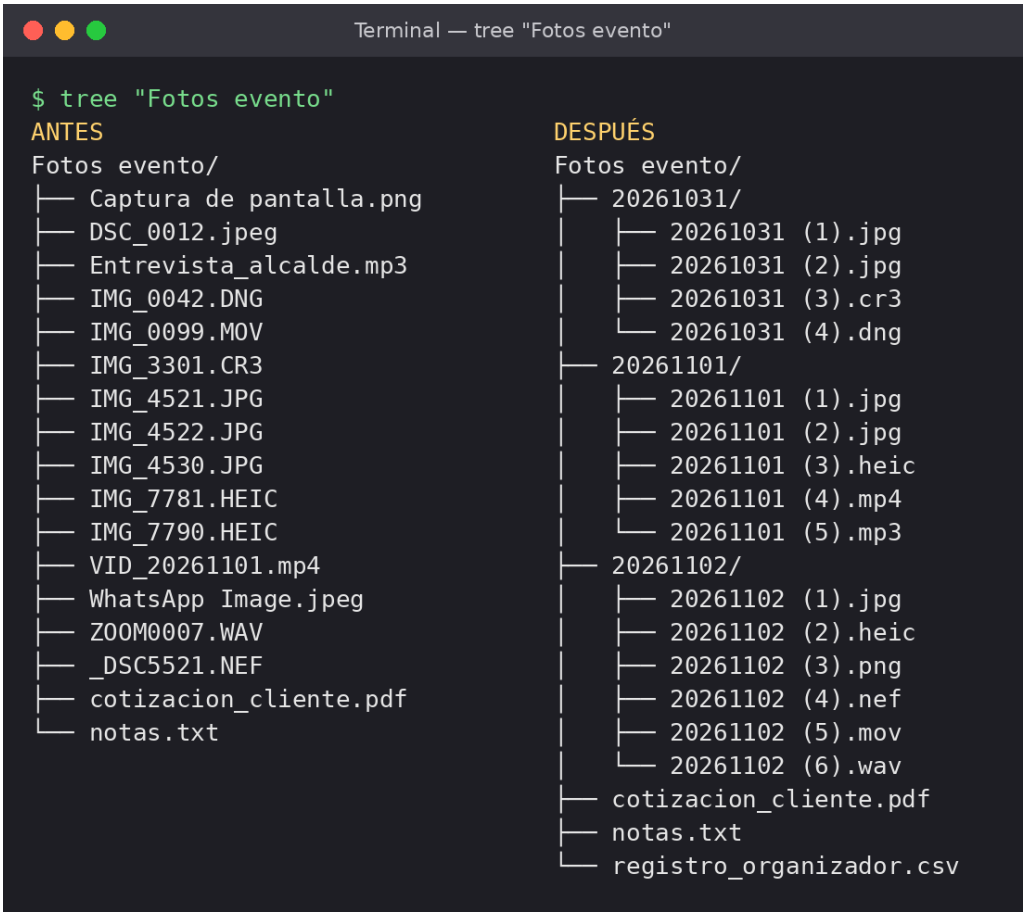
Se probó el programa con una carpeta llamada «Fotos evento» con 17 archivos: cuatro JPG de cámara, dos HEIC de iPhone, tres RAW (NEF, DNG y CR3), una captura PNG y una foto de WhatsApp sin datos EXIF, un video MP4 y uno MOV, una entrevista en MP3, un audio WAV de grabadora y dos archivos que no se organizan (un PDF y un TXT).



Captura 1. Vista previa y cancelación con «n»: no se modifica ningún archivo.



Captura 2. Ejecución completa: una respuesta inválida («quizas») hace que se repita la pregunta; con «s» se organizan los 15 archivos.

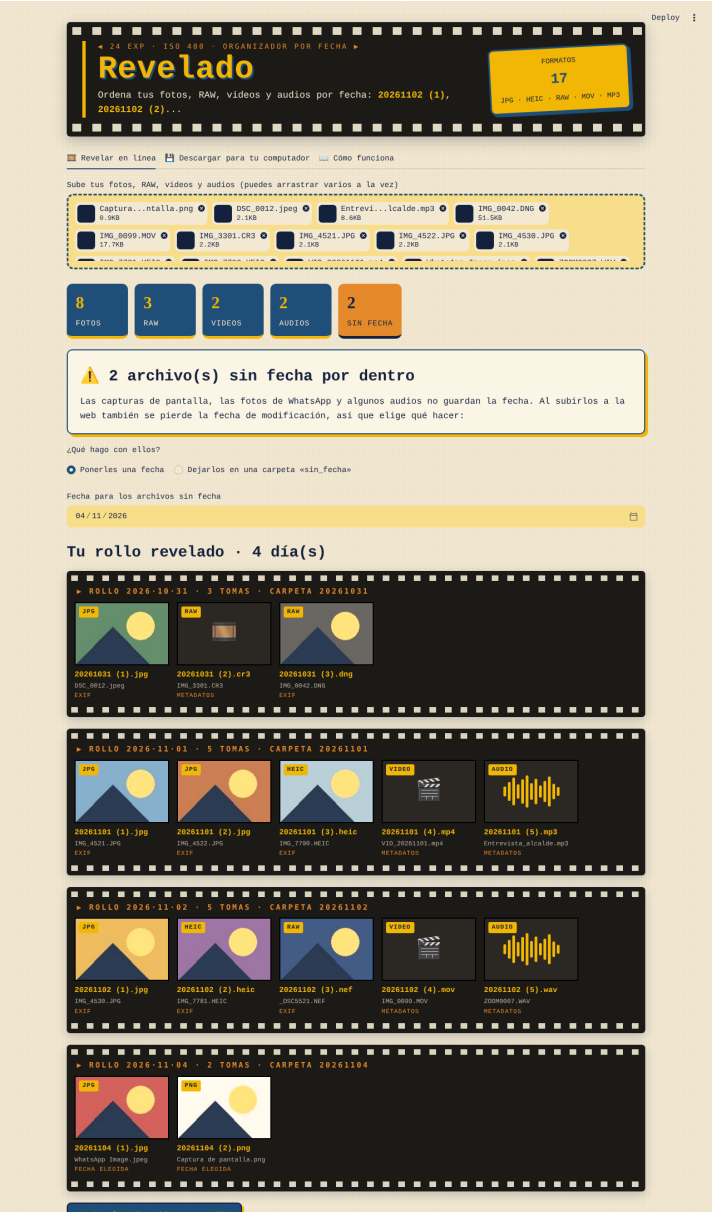


Captura 3. Contenido de la carpeta antes y después de ejecutar el programa.

En las pruebas se verifican todas las ramas del algoritmo: las fotos de cámara toman la fecha del EXIF; la captura PNG y la foto de WhatsApp usan la fecha de modificación; los HEIC y los RAW NEF y DNG leen su EXIF; el CR3 se lee como video; el MOV, grabado a las 9:45 p. m. del 2 de noviembre en Colombia, está guardado en UTC como 3 de noviembre y el programa lo ubica en el día 2; el WAV toma la fecha de la grabadora y el MP3 la de su etiqueta ID3; el PDF y el TXT se omiten. El 2 de noviembre muestra el orden completo: JPG (1), HEIC (2), PNG (3), RAW (4), video (5) y audio (6).

Versión web (Streamlit)

La misma lógica se publica como app web con una interfaz en forma de rollo fotográfico. Como la app corre en un servidor y no puede tocar las carpetas del computador, la persona sube los archivos, revisa el rollo de cada día y descarga un ZIP organizado; para carpetas grandes puede descargar el programa de escritorio desde la misma app.



Captura 4. App web: un rollo por día con las tomas ya renombradas y la fuente de cada fecha.

7. Reflexión individual

Elegí este problema porque lo vivo todas las semanas. En un solo cubrimiento puedo terminar con cientos de archivos: fotos de la cámara en JPG, capturas en PNG, videos del celular en MOV y clips en MP4, cada uno con nombres como IMG_4521 o VID_20261101 que no dicen nada. Buscar después el material de un evento es lento. Convertir esa tarea repetitiva en un algoritmo me mostró que la programación no es un tema aparte de mi oficio, sino una herramienta para ahorrarme horas de trabajo manual.

Lo primero que tuve que resolver no fue de código, sino de diseño: cómo trabajar con los archivos sin que nadie tuviera que subirlos a ninguna parte. La respuesta fue que el programa se ejecute en el propio computador y que el permiso sea un acto consciente del usuario: elegir la carpeta en una ventana del sistema. Aun así, cambiar el nombre de cientos de archivos es una acción delicada, así que agregué dos protecciones: una vista previa con confirmación, que es una decisión (if) dentro de un ciclo (while) que no avanza hasta recibir «s» o «n», y un registro CSV con los nombres originales por si hay que revertir algo.

Las estructuras de decisión aparecieron de forma natural. El programa debe tomar decisiones distintas según la extensión del archivo: si es foto, busca la fecha de toma en los datos EXIF; si es video, la busca dentro del archivo; si es otro tipo, lo deja quieto. Aprendí a normalizar la extensión a minúsculas antes de compararla, porque mi cámara guarda «.JPG» y el celular «.jpg», y sin ese detalle el if habría fallado en silencio. También aprendí que no todos los archivos traen la fecha por dentro: las capturas de pantalla y las fotos que llegan por WhatsApp pierden esos datos, por lo que el algoritmo necesita un plan B, la fecha de modificación del archivo.

Con los ciclos entendí mejor cuándo usar cada uno. El for recorre la lista de archivos porque sé cuántos hay. El while, en cambio, aparece cuando no sé cuántas repeticiones habrá: validar una carpeta, esperar una respuesta válida, buscar un número libre para no sobrescribir un archivo existente o recorrer los bloques internos de un video hasta encontrar el que guarda la fecha. Este último fue el mayor reto técnico: descubrí que un archivo MP4 o MOV está hecho de «átomos» y que la fecha de creación viene en hora UTC, así que un video grabado a las 9:45 p. m. en Colombia aparecía con fecha del día siguiente hasta que hice la conversión a la hora local.

Otro reto fue la numeración. Si dos fotos se tomaron el mismo día, no pueden llamarse igual. La solución fue ordenar los archivos por día y, dentro de cada día, por tipo: primero las fotos JPG, luego las PNG y al final los videos, cada grupo en el orden en que se tomó. Luego un contador por día asigna una sola secuencia continua, de modo que las fotos quedan como 20261101 (1), 20261101 (2) y el video del mismo día sigue como 20261101 (3). Esta decisión de orden también se resolvió con una estructura if / elif / else que le da una prioridad a cada tipo de archivo. Además, si el programa se ejecuta dos veces sobre la misma carpeta, el ciclo que revisa si el destino existe evita

que se pierda información. Pensar en esos casos límite antes de programar, con el pseudocódigo y el diagrama de flujo, me ayudó a encontrar errores que no habría visto escribiendo el código directamente.

Al revisar el informe decidí ampliar los formatos, porque en mi trabajo no solo hay JPG y videos: también llegan fotos HEIC del iPhone, archivos RAW de distintas cámaras y audios de entrevistas en WAV o MP3. Lo interesante fue que la estructura del algoritmo casi no cambió: bastó con agregar ramas a la cadena if / elif / else que clasifica cada archivo y otra a la que decide de dónde leer la fecha. Descubrí, por ejemplo, que el CR3 de Canon está construido igual que un video, así que pude reutilizar el mismo ciclo while que recorre los átomos de un MP4. Eso me mostró el valor de separar el programa en funciones pequeñas y reutilizables.

El paso a la web con Streamlit me obligó a pensar en otra limitación: una app publicada corre en un servidor y no puede tocar las carpetas de quien la abre. Por eso la versión web funciona subiendo archivos y descargando un ZIP organizado, y ofrece descargar el programa de escritorio para los cubrimientos grandes. Además, al subir un archivo se pierde su fecha de modificación, así que en la web tuve que agregar otra decisión: preguntarle al usuario qué fecha ponerles a los archivos que no la traen por dentro.

Como crítica a mi propia solución, reconozco algunos límites. El programa solo revisa la carpeta elegida y no sus subcarpetas, la lectura de fechas depende de librerías externas (exifread, Pillow, pillow-heif, mutagen) y los MP3 casi nunca guardan el día de grabación. Si el reloj de la cámara está mal configurado, el archivo quedará en el día equivocado. Además, cuando se dispara en RAW+JPG, los dos archivos de una misma foto reciben números distintos. Mejorarlo implicaría recorrer subcarpetas con recursión y emparejar archivos con el mismo nombre. Aun así, me quedo con lo más importante: detrás de herramientas que uso a diario, como el organizador de fotos del celular o de Lightroom, hay exactamente estas estructuras de decisión y repetición, y ahora puedo construir las mías a la medida de mi trabajo.